

Detailed Report: Code Analysis of 1_task.R

July 10, 2026

1 Introduction

This report provides a detailed, line-by-line analysis of the 1_task.R file, which corresponds to Task 1 of the project. The primary objective of the script is to estimate different Beta mixture models (ranging from 1 to 5 components) using Bayesian methods via JAGS and to select the optimal model based on the WAIC (Watanabe-Akaike Information Criterion).

2 Line-by-Line Code Analysis

2.1 Data and Structure Initialization

```
1 # =====
2 # TASK 1: Beta Mixture Models and WAIC Selection
3 # =====
4
5 # JAGS requires data in a list format
6 jags_data <- list(
7   y = y,
8   N = N
9 )
10
11 # Lists to store the results
12 waic_list <- list()
13 models_mcmc <- list()
```

- **Lines 1-5:** Introductory comments.
- **Lines 6-9:** A list named `jags_data` is created, containing the observations (`y`) and the total number of observations (`N`). JAGS requires input data to be structured as a list. We assume that `y` and `N` were defined previously (e.g., in `0_setup.R`).
- **Lines 11-14:** Two empty lists, `waic_list` and `models_mcmc`, are initialized. These are used to store the computed WAIC values and the MCMC (Markov Chain Monte Carlo) samples extracted for each estimated model, respectively.

2.2 JAGS Model Fitting Function Definition

```
15 # Function to run JAGS model for a given H
16 fit_beta_mixture <- function(H, data, n_iter = 500, n_burnin = 500) {
```

- **Line 16:** The `fit_beta_mixture` function is defined. This function takes as input the number of mixture components `H`, the data `data`, and the MCMC parameters (`n_iter` for sampling iterations and `n_burnin` for the warm-up phase).

2.2.1 Model for $H = 1$ (Single Beta)

```

18 if (H == 1) {
19   # Single Beta model
20   model_string <- "
21   model {
22     for (i in 1:N) {
23       y[i] ~ dbeta(alpha, beta)
24     }
25
26     # Priors (weakly informative)
27     alpha ~ dgamma(0.1, 0.1)
28     beta ~ dgamma(0.1, 0.1)
29   }
30   "
31   params_to_monitor <- c("alpha", "beta")

```

- **Lines 18-32:** Handles the case where the number of components is 1, representing a standard, non-mixture Beta model.
- **Lines 22-24:** The model likelihood: each observation $y[i]$ follows a Beta distribution with parameters `alpha` and `beta`.
- **Lines 27-28:** Prior distributions are defined for `alpha` and `beta`. Gamma(0.1, 0.1) distributions are used, which are weakly informative, allowing the data to drive the estimation.
- **Line 31:** Specifies the parameters we want JAGS to return and monitor: `alpha` and `beta`.

2.2.2 Model for $H > 1$ (Beta Mixture)

```

33 } else {
34   # Mixture of H Betas with ordering constraint on means to prevent label switching
35   model_string <- paste0("
36   model {
37     for (i in 1:N) {
38       y[i] ~ dbeta(alpha[z[i]], beta[z[i]])
39       z[i] ~ dcat(pi[1:H])
40     }
41
42     # Priors for mixing weights
43     pi[1:H] ~ ddirich(rep(1, H))
44
45     # Priors for Beta parameters using mu and precision (kappa)
46     # with ordering on mu to prevent label switching
47     for (h in 1:H) {
48       mu_raw[h] ~ dunif(0, 1)
49       kappa[h] ~ dgamma(0.1, 0.1)
50     }
51
52     # Sort mu_raw to ensure mu[1] < mu[2] < ... < mu[H]
53     mu[1:H] <- sort(mu_raw)
54
55     for (h in 1:H) {
56       alpha[h] <- mu[h] * kappa[h]
57       beta[h] <- (1 - mu[h]) * kappa[h]
58     }
59   }
60   ")
61
62   # Update data list to include H
63   data$H <- H
64   params_to_monitor <- c("alpha", "beta", "pi")
65 }

```

- **Lines 33-65:** If $H > 1$, the mixture model is defined.
- **Line 38:** Each observation y_i follows a Beta distribution whose parameters depend on the assigned component z_i .
- **Line 39:** The latent variable z_i (cluster assignment) follows a categorical distribution with probabilities given by the mixture weights `pi`.

- **Line 43:** A symmetric Dirichlet prior is used for the mixture weights `pi`, ensuring the weights sum to 1.
- **Lines 47-50:** To address the component identifiability issue (label switching), the Beta distributions are parameterized using the mean `mu` and precision `kappa`. Initially, raw means are generated from a Uniform(0,1) distribution, and precisions from a weakly informative Gamma.
- **Line 53:** The means are sorted in ascending order. This step is crucial for preventing label switching: it imposes a strict ordering on the components, ensuring the algorithm does not swap cluster labels during sampling.
- **Lines 55-58:** The canonical shape parameters of the Beta distribution, `alpha` and `beta`, are calculated from the sorted means `mu` and precisions `kappa`.
- **Lines 63-64:** `H` is added to the data list, and JAGS is instructed to monitor the parameters `alpha`, `beta`, and the weights `pi`.

2.2.3 JAGS Execution

```

67 # Write model to temporary file
68 model_file <- tempfile()
69 writeLines(model_string, con = model_file)
70
71 # Initialize and run JAGS
72 cat(paste("\nFitting model with H =", H, "... \n"))
73 jags_model <- jags.model(file = model_file, data = data, n.chains = 2, n.adapt = n_
    burnin, quiet = TRUE)
74
75 # Burn-in
76 update(jags_model, n_burnin)
77
78 # Sample
79 samples <- coda.samples(jags_model, variable.names = params_to_monitor, n.iter = n_
    iter)
80
81 return(samples)
82 }
```

- **Lines 68-69:** The model string is written to a temporary file for JAGS to read.
- **Line 73:** The JAGS model is initialized by compiling the code, using 2 chains (`n.chains = 2`) and starting an initial adaptation phase of length `n.burnin`.
- **Line 76:** A further burn-in of `n_burnin` iterations is executed to allow the chains to converge toward the stationary distribution before collecting final samples.
- **Line 79:** Posterior samples are drawn for `n_iter` iterations for the monitored variables. The result is returned by the function.

2.3 Model Estimation and Evaluation (Main Loop)

```

84 # Run models for H = 1, 2, 3, 4, 5
85 for (H in 1:5) {
86   n_it <- if(exists("GLOBAL_N_ITER")) GLOBAL_N_ITER else 500
87   n_bu <- if(exists("GLOBAL_N_BURNIN")) GLOBAL_N_BURNIN else 500
88
89   samples <- fit_beta_mixture(H, jags_data, n_iter = n_it, n_burnin = n_bu)
90   models_mcmc[[H]] <- samples

```

- **Line 85:** A loop begins to test models with a number of components `H` ranging from 1 to 5.
- **Lines 86-87:** The number of iterations and burn-in values are set. If global variables for these values exist (presumably defined in `0_setup.R`), they are used; otherwise, it defaults to 500.
- **Lines 89-90:** The `fit_beta_mixture` function is called, and the returned samples are saved in the `models_mcmc` list.

2.3.1 Log-Likelihood Calculation for WAIC

```
92 # Extract and compute log_lik for WAIC
93 samples_mat <- as.matrix(samples)
94 S <- nrow(samples_mat)
95 log_lik_mat <- matrix(0, nrow = S, ncol = N)
96
97 cat("Computing WAIC...\n")
98 for (s in 1:S) {
99   if (H == 1) {
100     log_lik_mat[s, ] <- dbeta(y, samples_mat[s, "alpha"], samples_mat[s, "beta"], log
= TRUE)
101   } else {
102     lik_i <- rep(0, N)
103     for (h in 1:H) {
104       pi_h <- samples_mat[s, paste0("pi[", h, "]")]
105       alpha_h <- samples_mat[s, paste0("alpha[", h, "]")]
106       beta_h <- samples_mat[s, paste0("beta[", h, "]")]
107       lik_i <- lik_i + pi_h * dbeta(y, alpha_h, beta_h, log = FALSE)
108     }
109     log_lik_mat[s, ] <- log(lik_i)
110   }
111 }
```

- **Lines 93-95:** The MCMC object is converted into a matrix of S rows (total samples across all chains). A `log_lik_mat` matrix of dimensions $S \times N$ (samples by observations) is initialized to hold the pointwise log-likelihood.
- **Line 98:** Iterates over each MCMC sample s .
- **Line 100:** If $H = 1$, the log-likelihood for each observation is calculated in a vectorized manner using the α and β parameters extracted from that sample.
- **Lines 101-110:** If $H > 1$, the observation likelihood is the weighted sum of the individual mixture component likelihoods. For each component h , it calculates $\pi_h \times \text{Beta}(y \mid \alpha_h, \beta_h)$ and sums them up. Finally, at line 109, the logarithm is applied to the total likelihood of the mixture.

2.3.2 WAIC Calculation and Model Selection

```
113 # Calculate WAIC using loo package
114 waic_res <- waic(log_lik_mat)
115 waic_list[[H]] <- waic_res
116
117 cat(sprintf("WAIC for H = %d: %.2f\n", H, waic_res$estimates["waic", "Estimate"]))
118 }
119
120 # Select best model based on WAIC
121 waic_values <- sapply(waic_list, function(w) w$estimates["waic", "Estimate"])
122 best_H <- which.min(waic_values)
123
124 cat(sprintf("\nBest model selected by WAIC has H = %d components (WAIC = %.2f).\n", best
_H, waic_values[best_H]))
```

- **Lines 114-115:** The `waic()` function (presumably from the `loo` package) takes the log-likelihood matrix `log_lik_mat` as input and calculates the WAIC index, then saves it in the `waic_list`.
- **Lines 120-122:** After the loop concludes, the estimated WAIC value for each model is extracted, and the index (`best_H`) corresponding to the model that minimizes the WAIC is found (a lower value indicates a better trade-off between goodness of fit and model complexity).

2.4 Saving the Results

```
126 # Save results for subsequent tasks
127 best_model <- best_H
128 waic_results <- waic_list
129 mcmc_samples <- models_mcmc[[best_H]]
```

- **Lines 126-129:** The number of components for the optimal model, the complete WAIC results, and the MCMC samples of the selected best model are saved into specifically named variables (`best_model`, `waic_results`, `mcmc_samples`). This ensures that this essential information is available in the global environment to be reused by subsequent scripts (e.g., `2_task.R`).

3 Conclusion

The code is well-structured. A highly important feature is the ordering of the means (`mu`) in the JAGS block to prevent the "label switching" problem, an excellent practice in Bayesian modeling of mixtures. The pointwise log-likelihood calculation (necessary for WAIC) is correctly implemented by converting the JAGS samples into a matrix and extracting the log-likelihood pointwise for each observation and each MCMC iteration.